ASignX system report

21 October 2026 | Architecture, data, evaluation and development priorities

ASignX is a working hand-landmark recognition prototype with static alphabet classification, experimental J/Z motion recognition, and substantial data collection tooling. Its main limitation is generalization: static validation accuracy is **97.64%**, while test accuracy is **53.19%**. The dynamic model's **100%** test result covers only **two clips**.

Verification for this report: source inspection, loading both retained checkpoints, a current static dataset recount, numerical/schema and split validation, and a fresh full test run. Recognition scores are from saved evaluation artifacts; complete recognition evaluation and training were not rerun on 21 October.

1. Current system architecture

The application runs locally in Python. OpenCV handles capture and display, MediaPipe detects hand landmarks, NumPy prepares features, and PyTorch classifies signs.

Stage	Processing
Input	Webcam frames, images or recorded videos
Detection	MediaPipe hand detection/tracking; 21 landmarks per hand and handedness
Preparation	Normalization, feature enrichment and timestamped motion routing
Classification	Static pose classifier or dynamic sequence classifier
Decision/output	Confidence filtering, temporal stabilization, displayed labels and recognition events

Component	Status
Camera and detector	Threaded capture, exposure controls and tracking of up to two hands implemented
Recognition	24 static labels plus dynamic J and Z
Data tools	Collection, image/video extraction, CSV merging and recording GUIs implemented
Training/evaluation	Split manifests, integrity checks, metrics and error audits implemented
Browser SDK	TypeScript preprocessing, motion and stabilization helpers; integration remains partial

main.py starts landmark tracking by default; recognition requires model arguments. The demonstrated trained pipeline uses hands. Facial expression, body-relative meaning, grammar and sentence context are outside its demonstrated scope.

Repository evidence: main.py; asignx/camera.py; asignx/detector.py; asignx/visualizer.py; web_client/asignx_client_sdk.ts

2. Model architecture and algorithms

Property	Static model	Dynamic model
Vocabulary	A-I and K-Y; 24 labels	J and Z
Input	63 values per hand pose	30 timesteps x 126 values
Enriched input	91 features	182 features per timestep
Backbone	Residual MLP; widths 128 and 64	128-wide projection; two residual temporal-convolution blocks; four-head self-attention
Pooling/head	Cosine prototype head	Mean temporal pooling; 64-dimensional embedding; cosine prototype head
Parameters	63,681	296,641
Regularization	LayerNorm; dropout 0.2	LayerNorm; dropout 0.2

The retained dynamic checkpoint uses the tcn_attention backbone, despite being loaded through a class named SignLSTM. An LSTM alternative is implemented but is not the backbone of that checkpoint. Kinematic enrichment adds 15 joint-angle cosines, 10 fingertip distances and a three-component palm normal per hand.

Preprocessing and inference

Static preprocessing subtracts the wrist position and divides coordinates by the maximum landmark distance from the wrist. The canonical checkpoint additionally reflects the x coordinates of detector-labelled Left hands. CSV coordinates retain their original noncanonical representation.

Dynamic preprocessing uses a clip-wide reference and scale to preserve wrist travel and relative hand positions. Missing observations are interpolated; complete gestures are resampled to 30 steps. Live inference uses timestamped sampling, motion routing, confidence filtering and repeated-prediction stabilization. The retained confidence threshold is 0.8. Softmax confidence is not a validated probability of gesture correctness.

Training configuration

Both saved models used AdamW, weight decay 0.0001, cosine learning-rate annealing, inverse-frequency class weights, cross-entropy with 0.05 label smoothing, small rotation/jitter augmentation, seed 42, CPU training and a 50-epoch schedule. Static training used batch size 64 and learning rate 0.002; dynamic training used batch size 16 and learning rate 0.0015.

The retained checkpoints were selected by validation loss. Current trainers default to validation macro F1 with loss as a tie-breaker, and support tuning without test evaluation.

Repository evidence: ml/model.py; ml/sequence_model.py; ml/preprocessing.py; ml/predict.py; ml/predict_sequence.py; ml/train.py; ml/train_sequence.py; retained checkpoint metadata

3. Data models and dataset profile

Data model	Contents
Landmark	ID, anatomical name and x/y/z coordinates
HandLandmarks	Exactly 21 landmarks, handedness and handedness confidence
HandDetectionResult	Detected hands, left/right accessors and frame timestamp
Static Sample	Label, session ID, handedness and 63 float features
SequenceSample	Label, session ID, handedness, T x 63 or T x 126 features, and metadata
Split manifest	Dataset hash, preprocessing version, labels, seed and partition indices
Checkpoint	Weights, architecture configuration, labels, preprocessing and training metadata

Static CSV records contain four metadata columns plus 63 coordinate columns. Dynamic samples use compressed .npz files. Two-hand sequences use stable Left then Right feature slots.

Current data quantities

Measure	Static dataset	Retained dynamic baseline
Samples	159,987 rows	11 clips
Labels	24	2
Training	109,223	7
Validation	13,791	2
Test	36,973	2
Session/identity	1,639 session strings	Signer independence unverified

Static handedness counts are 98,909 Right and 61,078 Left. Class counts range from 5,281 for N to 8,252 for L, a 1.56x imbalance. Split fractions apply to session groups, so row counts differ substantially from a 70/15/15 allocation. Uneven group sizes and source distributions matter more here than overall class imbalance.

The portable data folder contains 16 dynamic clips: 11 J and 5 Z. Eleven reproduce the retained baseline; the five additional clips are all J. The retained 11-clip baseline has 6 J and 5 Z clips.

The current static CSV hash matches the retained baseline. SHA-256:
17249bc657b7ed52f1d6a39f5d3bbbf6b8d2445efdab97f64094df68855647db

Repository evidence: asignx/models.py; ml/data_io.py; ml/sequence_io.py; ml/data/landmarks.csv; ml/models/*.splits.json; output/f1-review/dynamic/recovery.json

4. Recognition performance and evaluation

Model	Validation accuracy	Validation macro F1	Test accuracy	Test macro F1
Retained static	97.64%	97.33%	53.19%	59.49%
Retained dynamic	100%	100%	100%	100%

These are unthresholded classification results. Dynamic validation and test each contain only one J and one Z clip. The static test contains 36,973 rows; dynamic test contains two clips. Their percentages do not carry comparable evidential weight.

Static error patterns

The static validation-to-test accuracy gap is **44.46 percentage points**. Test weighted F1 is **56.16%**. Weak test F1 classes include P (18.56%), G (21.71%), N (22.81%), V (23.44%) and D (32.88%).

Recorded confusion	Count
K to V	3,304
M to N	2,437
Q to P	2,280
G to L	1,870
G to H	1,711

The saved audit reports 99.93% accuracy on archive5-hint rows versus 37.82% on hashed-import rows. These strings are source hints, not verified identities. The difference supports source-specific investigation; it does not prove whether labels, acquisition conditions or preprocessing caused the errors.

Dynamic evaluation limitation

The two-clip test remains perfect after time reversal, frame shuffling, last-frame repetition or removal of wrist travel. **This benchmark does not demonstrate movement understanding.** Unknown signer identities and the very small sample size prevent a reliable generalization claim.

Controlled experiments and retained configuration

Five controlled static training variants failed to improve the incumbent. The validation-selected candidate achieved 54.98% test macro F1, below the retained model's 59.49%. These were short 12-epoch screens, compared with the incumbent's 50-epoch run. The recorded decision retains the canonical static model and threshold 0.8.

A validation-fitted threshold of 0.535 achieved 61.54% historical test macro F1, versus 63.06% at threshold 0.8 with rejected known signs counted as false negatives. Neither threshold has representative unknown-gesture validation.

Repository evidence: ml/models/*.metrics.json; output/f1-review/baseline/summary.json; output/f1-review/dynamic/reproduced.json; output/f1-review/release-decision.json; docs/f1-workflow.md

5. Validation, runtime and software quality

Data integrity checks

Existing checks validate schemas, finite coordinates, normalization, preprocessing versions, label mappings, dataset hashes and partition coverage. They keep sessions within one partition and reject identical feature fingerprints across partitions. The current CSV passed numerical/schema validation and canonical split validation on 21 October.

Protection	Remaining limitation
Session-disjoint partitions	A session string does not prove an independent signer or recording
Exact-feature duplicate checks	Near duplicates and related-source dependence can remain
Numerical and schema validation	Valid coordinates do not establish a linguistically correct label
Dataset/checkpoint hashes	Reproducibility does not establish representative data coverage
Explicit global identity plans	Real provenance is required; historical identities remain unknown

Historical splits do not establish signer independence across labels. Global identity plans now keep supplied signer, recording and source identities together across labels, but must be populated from verified provenance. Representative idle, transition and incorrect-gesture evaluation is still missing.

Fresh software verification

108 tests passed, with 13 warnings, in the full test run on 21 October 2026. Warnings concern export tracing and deprecated TorchScript tracing APIs. Both retained checkpoints loaded successfully, and parameter counts were verified. Passing software tests does not resolve weak held-out recognition accuracy.

Runtime and data performance

Available evidence does not establish sustained camera FPS, end-to-end latency, detector recall, memory usage or continuous-stream false activations. Camera FPS settings are targets rather than measured throughput. Dataset volume alone does not measure independent signer coverage.

Live safeguards include timestamped sampling, camera-stall resets, poor-visibility rejection and both-hand motion routing. Complete-gesture training still differs from sliding-window inference. Static hold/release counts remain frame-based, so their effective duration depends on frame rate.

Browser deployment

The browser helpers provide a foundation, but a verified end-to-end camera/model deployment remains incomplete. The exporter can fall back to TorchScript after ONNX export failure; this fallback does not establish ONNX Runtime Web compatibility. Export success and runtime output parity need explicit verification.

Repository evidence: ml/dataset.py; ml/benchmark.py; ml/preprocessing.py; ml/predict_sequence.py; ml/export_onnx.py; web_client/asignx_client_sdk.ts; tests/

6. Recommended next steps

1. Audit the largest static errors by original source

Verify labels, mirror conventions and handedness for K/V, M/N, Q/P and G/L/H. Record evidence-backed corrections and preserve legitimate difficult examples.

2. Build a provenance-backed benchmark

Keep actual signers and related recordings together across all labels. Reserve a fresh untouched final test set before further model selection.

3. Collect diverse dynamic data and negatives

Include different signers, visits, speeds and devices, plus matching-handshape holds, incorrect paths, idle and transitions. The existing 10-person checklist is a planning starting point, not a guarantee of statistical adequacy.

4. Evaluate continuous recordings

Measure event precision/recall, false emissions per minute, repeated-sign handling, missed gestures and decision delay. Include real occlusions and tracking failures.

5. Profile the complete runtime

Measure capture, detection, preprocessing, classification and rendering separately. Report median and tail latency, sustained FPS and device details.

6. Repeat controlled training on reviewed data

Select settings using grouped validation and evaluate the final test only after selection. Compare per-class and source-specific results as well as aggregate F1.

7. Complete deployment verification

Require successful ONNX export, compare Python/browser outputs, and rebuild packaged collection tools to include current identity metadata.

Supporting repository reports

docs/system-state-current.md and output/pdf/system-state-current.pdf contain the longer 20 October review. docs/f1-workflow.md documents reproduction commands, controlled experiments, identity plans and collection readiness. Saved metrics and audit artifacts are under ml/models/ and output/f1-review/.

This PDF records the report delivered in the conversation, with the same measured findings and limitations. No models were retrained and no dataset rows were changed to produce it.